

Replication in Kafka

[Aman Arora](#)



Welcome to the second part of the series about Apache Kafka.

In the first part, we learned about some of the basic terminologies in Kafka like topics, partitions, brokers etc and in this one, I'll be writing about Replication in Kafka.

If you haven't read the [first part](#), do check it out.

What is Replication?

As mentioned in the first part of the series — we use Kafka as a cluster, meaning that data in Kafka is spread across multiple servers.

Replication is the process of having multiple copies of the data for the sole purpose of availability in case one of the brokers goes down and is unavailable to serve the requests.

In Kafka, replication happens at the partition granularity i.e. copies of the partition are maintained at multiple broker instances using the partition's write-ahead log.

Every partition in a topic has a write-ahead log where all the messages for that partition are stored in order. The messages are identified by the unique offset.

Replication factor defines the number of copies of the partition that needs to be kept.

A Kafka cluster with Replication Factor 2

A replication factor of 2 means that there will be two copies for every partition.

Leader for a partition: For every partition, there is a replica that is designated as the **leader**. The Leader is responsible for sending as well as receiving data for that partition. All the other replicas are called the **in-sync replicas** (or followers) of the partition.

In-sync replicas are the subset of all the replicas for a partition having same messages as the leader.

Now let's see what happens when a broker goes down. If for some reason let's say Broker 2 goes down. The access to partition 1 is now lost since broker 2 was the leader for partition 1. What happens now is Kafka automatically selects one of the in-sync replicas (in our case

there is only 1 replica) and makes them the leader. Now when broker 2 comes back online, it can try to become the leader again.

Now that we know about replication in Kafka, there is one thing that was left out in part I of the series.

Producers can choose to receive acknowledgements for the data writes to the partition using the “acks” setting.

There are 3 levels of acknowledgements that Producers can choose from depending upon their use case of Kafka.

The value of *acks* varies from application to application. For an application where high durability needs (example in case of transaction data), *acks = all* is recommended whereas in cases where lower latency is more important *acks = 0* is used (example — user’s location data).

Note: *Acks parameter defines the number of acknowledgements that should be waited for from the in-sync replicas only.*

How does Replication Work?

In Kafka, the following 2 conditions need to be met for a node to be considered alive.

- The node must maintain its session with zookeeper
- If the node is a follower, it must not be “*far behind*” the leader

Naturally, you must be asking how does a leader determine if a follower is caught up or not?

The answer is fairly simple — leader maintains a list of its followers and tracks their status. If a follower dies or is not able to replicate and falls behind the leader, it gets removed from the in-sync replica list.

Kafka gives us the power to decide the condition for which a replica is considered stuck or falling behind.

- *replica.lag.max.messages*

This parameter decides the allowed difference between replica's offset and leader's offset. if the difference becomes more than (*replica.lag.max.messages-1*) then that replica is considered to be lagging behind and is removed from the in-sync replica list.

If the value of *replica.lag.max.messages* is *n*, it means that as long as the follower is behind the leader by not more than *n-1* messages, it won't be removed from the in-sync replica list.

- *replica.lag.time.max.ms*

This parameter defines the maximum time interval within which every follower must request the leader for its log. If for some reason a replica is unable to do so, it will be removed from the in-sync replica list.

Kafka guarantees that if a message has been acknowledged as committed, then even in case of leader failure, messages won't get lost.

Failure Recovery

Now that we know the basics of replication in Kafka, lets discuss how Kafka behaves in case of failures. We already know that — In Kafka, both reads and writes occur at leader. So, what happens in case Leader goes down? Well, a new leader needs to be chosen from the remaining replicas.

Only the replicas that are part of in-sync replica list are eligible for

becoming the leader and the list of in-sync replicas is persisted to zookeeper whenever any changes are made to it. Also, Kafka's guarantee of no data loss is applicable only if there exists an in-sync replica. In case of no such replica, this guarantee is not applicable.

Although, Its highly uncommon for all the replicas of a partition to go down. But in case this happens — there are 2 behaviours that can be implemented in Kafka —

- Wait for an in-sync replica to come up — in this case we chose the replica as leader and hope that it has all the data.
- Wait for any replica to come up

The choice really boils down to choosing between availability and consistency. *By default, the first behaviour is the chosen one in Kafka.*

Reasons for Replica's being behind?

A follower replica might be lagging behind the leader for a number of reasons.

1. **Slow Replica:** It is possible for a replica to be unable to cope up with the speed at which leader is getting new messages — the rate at which the leader is getting messages is more than the rate at which the replica is copying messages causing IO bottleneck.
2. **Stuck Replica:** If a replica has stopped requesting the leader for the new messages for reasons like a dead replica or the replica is blocked due to GC (Garbage collector).

I hope you found this blog helpful and insightful. Thanks for reading...

You can follow me on LinkedIn by clicking [here](#)

Peace out!

